Stack Project - Infix and Postfix Expressions

Project Report Template

Student: Emory Bruington

Course: CSC 223 - Data Structures and Analysis of Algorithms

GitHub Repository: https://github.com/l4mont/csc223-stack-infix-postfix

1. Project Description and Organization

In your own words, briefly describe what the program does, why a stack is appropriate for this problem, and how the major modules work together. Do not copy the project instructions.

Response:

The program converts ordinary infix expressions into postfix form and evaluates numeric postfix expressions. Stack provides the LIFO operations. The converter uses it for pending operators, and the evaluator uses it for operands and intermediate results. main.py imports both classes and demonstrates successful expressions and handled errors.

2. ADT Specification - Infix-to-Postfix Converter

Model

Briefly describe what an object of this class represents.

Model:

A reusable converter that translates an infix string into a space-separated postfix string. Each conversion has its own operator stack and does not retain expression state.

Main Conversion Operation

Operation/signature: convert(self, expression)

Purpose: Convert infix notation while preserving precedence, left associativity, and parentheses.

Input: A string of identifiers or unsigned integer/decimal operands, binary + - * /, parentheses, and optional whitespace.

Preconditions: The argument must be a string. The method checks expression syntax; unary signs and implicit multiplication are outside scope.

Postconditions/return value: Return a postfix string with single spaces between tokens. All pending operators are processed, and no parentheses remain.

Error conditions: TypeError for non-string input; ValueError for invalid characters, mismatched parentheses, misplaced operators/operands, empty input, or incomplete expressions.

Include the major errors that your implementation detects, such as invalid tokens, mismatched parentheses, misplaced operators or operands, and incomplete expressions.

Important Supporting Operation

Identify one important helper operation used by your converter.

Operation/signature: _tokenize(expression)

Purpose: Scan every character into tokens and report the position of an invalid character instead of silently ignoring it.

3. ADT Specification - Postfix Evaluator

Model

Briefly describe what an object of this class represents.

Model:

A reusable evaluator that computes the value of a numeric postfix string. Each call uses a new operand stack, so a failed call cannot affect the next expression.

Main Evaluation Operation

Operation/signature: evaluate(self, expression)

Purpose: Evaluate numeric postfix notation with a stack.

Input: A whitespace-separated string of unsigned integer/decimal operands and binary + - * / operators.

Preconditions: The argument must be a string. Tokens must follow the documented grammar; the method validates operand counts and syntax.

Postconditions/return value: Return the single int or float result. Division uses true division. Negative results are allowed.

Error conditions: TypeError for non-string input; ValueError for empty input, invalid tokens, insufficient or extra operands, or non-finite values/results; ZeroDivisionError for a zero divisor.

Include the major errors that your implementation detects, such as invalid tokens, insufficient operands, extra operands, malformed expressions, and division by zero.

Important Supporting Operation

Identify one important helper operation used by your evaluator.

Operation/signature: _parse_number(token)

Purpose: Check a numeric token and return an integer or finite float, raising ValueError if it cannot be accepted.

4. Testing and Debugging

Thoroughly test the individual modules while developing the program. You do not need to reproduce every test in this report. Select representative tests that demonstrate that your program handles both normal and erroneous input.

Module	Test Input or Operation	Expected Result	Actual Result	Pass/Fail
Stack	push 10, 20, 30; pop three times	30, 20, 10; empty	30, 20, 10; empty	Pass
Stack	pop() and peek() on empty stack	IndexError; size stays 0	IndexError; size 0	Pass
Infix Converter	A + B * C	A B C * +	A B C * +	Pass
Infix Converter	(A + B) * C; A - B + C	A B + C *; A B - C +	Both matched	Pass
Infix Converter - Error Case	A + * B	ValueError: misplaced operator	ValueError as expected	Pass
Postfix Evaluator	35 27 + 3 *	186	186	Pass
Postfix Evaluator	8 2 /; 2 8 -	4.0; -6	4.0; -6	Pass
Postfix Evaluator - Error Case	8 0 /; 2 3 4 +	ZeroDivisionError; ValueError	Both errors as expected	Pass
Integrated Program	main.py valid and error examples	16 expected output lines	All 16 lines matched	Pass

Your selected tests should provide evidence of several of the following where appropriate: LIFO stack behavior, operator precedence, parentheses, subtraction or division operand order, multi-digit operands, malformed expressions, and error processing.

Debugging Example

What was the problem or incorrect behavior?

The background handout uses a strictly-greater precedence comparison, which would mishandle equal-precedence operators in A - B + C.

What caused it?

Using > leaves subtraction on the stack when addition arrives. That produces A B C + -, which means A - (B + C), instead of the required left-to-right calculation.

How did you correct it?

The project follows the official instructions and pops operators with greater OR EQUAL precedence. A separate comparison reproduced the handout behavior and confirmed the difference.

What test showed that the correction worked?

The associativity test expects A - B + C to produce A B - C +. It passed. A / B * C also produced A B / C *, confirming the same rule for division and multiplication.

5. GitHub Development

List three or four meaningful commits that illustrate the development of your project.

Commit Message	What Was Completed or Changed
Implement and directly test the Stack ADT	fcf301b: Stack implementation and 10 direct test cases.
Implement postfix evaluation with validation and direct module tests	be92be1: Numeric evaluation and 44 direct test cases, including error handling.
Implement shunting-yard conversion and test syntax and associativity	7feb5f9: Shunting-yard conversion and 52 direct test cases.
Integrate converter and evaluator and verify demonstration output	62535f6: Integrated demonstration and independent output checks.

Briefly explain how your commits show that you developed and tested the program in stages rather than creating one final commit.

Response:

The Stack was implemented and tested first, followed by the postfix evaluator, the infix converter, and integration. Each completed stage was committed locally and published to GitHub before the next implementation stage began. The table lists the corresponding remote commits.

6. Reflection and Analysis

Answer the following four questions. A short paragraph for each question is sufficient when it fully explains your reasoning.

Question 1 - Stack and ADT Design

Why is a stack appropriate for both infix-to-postfix conversion and postfix evaluation? Also explain one advantage of having the converter and evaluator use the Stack interface rather than manipulating a Python list directly.

Response:

Conversion needs a stack because the most recent opening parenthesis or pending operator must be handled first. Evaluation needs one because an operator uses the most recently

available operands. The Stack interface keeps this behavior consistent and lets the underlying storage change without rewriting both algorithms.

Question 2 - Error Processing

Choose one malformed expression that you tested. Explain what was wrong with the expression, how your program detected the problem, and why reporting the error is preferable to allowing the program to continue.

Response:

The malformed expression A + * B has two operators with no operand between them. After reading +, the converter expects an operand, so * raises ValueError with a misplaced-operator message. Rejecting the expression prevents an incorrect postfix string from being passed on as a valid result.

Question 3 - Development and Testing

How did testing the Stack, PostfixEvaluator, and InfixToPostfixConverter separately help when you integrated the modules in main.py? Use at least one example from your development process.

Response:

Separate tests made it easier to locate problems. Stack passed its 10 test cases before the evaluator was added. The evaluator tests checked operand order and division by zero before conversion was integrated. When main.py combined the classes, 12 + 3 * (4 - 2) converted to 12 3 4 2 - * + and evaluated to 18.

Question 4 - Efficiency and Robustness

An expression contains n tokens. Explain why processing an expression with the stack-based conversion or evaluation algorithm is approximately linear in the number of tokens. Then briefly explain how input validation and error processing make the program more robust.

Response:

Each token is read once. Each operator or operand is pushed and popped only a limited number of times, so conversion and evaluation take approximately O(n) time under ordinary arithmetic-cost assumptions. Validation catches bad tokens, missing operands, mismatched parentheses, and zero divisors before they can produce misleading results. The program reports expected errors and continues with the next example.